Step 1: Install and Import Required Libraries

```
!pip install gensim nltk matplotlib
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.0)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 47.9 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

Step 2:Libraries import

```
import gensim
import gensim.downloader as api
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

Step 3:Load Pre-trained Word Embedding Models

```
word2vec_model = api.load("word2vec-google-news-300")
glove_model = api.load("glove-wiki-gigaword-100")
```

```
[==================================================] 100.0% 1662.8/1662.8MB downloaded
[==================================================] 100.0% 128.1/128.1MB downloaded
```

Step 4: Explore Word Vectors

```
print("Word2Vec vector dimension:", word2vec_model.vector_size)
print("GloVe vector dimension:", glove_model.vector_size)

print("\nVector for word 'king' (Word2Vec):")
print(word2vec_model["king"][:10])
```

```
Word2Vec vector dimension: 300
GloVe vector dimension: 100

Vector for word 'king' (Word2Vec):
[ 0.12597656  0.02978516  0.00860596  0.13964844 -0.02563477 -0.03613281
  0.11181641 -0.19824219  0.05126953  0.36328125]
```

Step 5: Compute Cosine Similarity Between Words

```
def similarity(model, w1, w2):
    v1 = model[w1].reshape(1, -1)
    v2 = model[w2].reshape(1, -1)
```

```
    return cosine_similarity(v1, v2)[0][0]

print("Similarity (king, queen) - Word2Vec:", similarity(word2vec_model, "king", "queen"))
print("Similarity (man, woman) - GloVe:", similarity(glove_model, "man", "woman"))
```

```
Similarity (king, queen) - Word2Vec: 0.6510957
Similarity (man, woman) - GloVe: 0.8323495
```

Step 6: Nearest Neighbors (Semantic Similarity)

```
print("Nearest words to 'computer' (Word2Vec):")
print(word2vec_model.most_similar("computer", topn=5))

print("\nNearest words to 'money' (GloVe):")
print(glove_model.most_similar("money", topn=5))
```

```
Nearest words to 'computer' (Word2Vec):
[('computers', 0.7979379892349243), ('laptop', 0.6640493273735046), ('laptop_computer', 0.6548868417739868), ('Computer', 0.6473

Nearest words to 'money' (GloVe):
[('funds', 0.8508071303367615), ('cash', 0.848483681678772), ('fund', 0.7594833374023438), ('paying', 0.7415367364883423), ('pay
```

Step 6: Analogy Task (king – man + woman = ?)

```
result = word2vec_model.most_similar(
    positive=["king", "woman"],
    negative=["man"],
    topn=1
)

print("Analogy result (king - man + woman):", result)
```

```
Analogy result (king - man + woman): [('queen', 0.7118193507194519)]
```

Step 7:Visualize Word Embedding Neighborhood (Optional but Recommended)

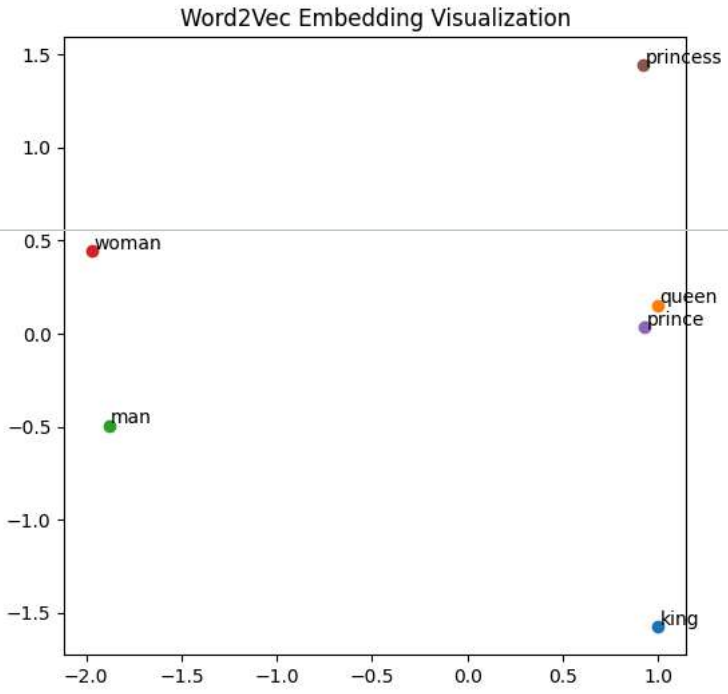
```
words = ["king", "queen", "man", "woman", "prince", "princess"]

vectors = np.array([word2vec_model[word] for word in words])

pca = PCA(n_components=2)
reduced = pca.fit_transform(vectors)

plt.figure(figsize=(6,6))
for i, word in enumerate(words):
    plt.scatter(reduced[i,0], reduced[i,1])
    plt.text(reduced[i,0]+0.01, reduced[i,1]+0.01, word)

plt.title("Word2Vec Embedding Visualization")
plt.show()
```


Word2Vec Embedding Visualization
princess
woman
queen
prince
man
king
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0